Overview of a classification benchmark

Alban Bourbon, Université Paris-Dauphine – PSL

2023, June 16 at Inria Saclay

Supervised by Thomas Moreau

Abstract

The purpose of my internship was to upgrade an existing classification *benchmark*[1]. At the beginning this benchmark contained only one dataset and two solvers – LinearRegression and SVM. Over the weeks, I added three new solvers (RandomForest, HistGradientBoostingClassifier et XGBoost) along with two files for new datasets. The first file contain the Iris dataset[2], which is directly implemented in scikit-learn. The second one, load several datasets from Léo Grinsztajn (2022) paper. I was inspired by his paper for the parameters range selection of the solvers as well as for the preprocessing step coming from his code used for his paper[3]. Optimization methods were then implemented firstly with hyperparameters optimization using Optuna package then by creating a VotingClassifier[4]-like.

[1] `https://github.com/tomMoral/benchmark_classification`
[2] Available here *sklearn.datasets*
[3] `https://github.com/LeoGrin/tabular-benchmark`
[4] `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html`

Datasets

In addition of the existing dataset, adding `Iris` dataset and Grinsztajn (2022) datasets allows to compare the solvers on known dataset and the results obtained, with those of Grinsztajn (2022).

Simulated : This dataset was created artificially in order to represent a perfectly linear data. To do so, the seed – which controls the way data are generated – takes the value 27 then create a random number for the `random_state` of Benchopt[5] method `make_correlated_data`. Simulated is set to take 1000 samples with 500 features or 5000 samples with 200 features. I also created a variable called `cat_indicator` for the preprocessing step.

Iris : sklearn's `load_iris` gives me directly train data, I just simply added the `cat_indicator` variable.

OpenML datasets: These datasets were chosen and modified by Grinsztajn (2022) so the features are of various kinds and the data not too small or artificial. The file containing these datasets has also a `cat_indicator` variable.

Optimization

The aim of optimizing is to make sure that the solvers are well generalized for the objective. This require using various methods such as hyperparameters selection or data modifications in order to make them understandable by the solver.

GridSearch : The grid search let us to take list of hyperparameters and to test it on solvers. The job of `GridSearch` is going to be to return the best hyperparameters combination i.e the one that gives the best score.

RandomSearch : Unlike GridSearch, RandomSearch will choose a random real in a defined interval. The interval is defined with hyperparameters suggestions made byOptuna and reals can be from a logarithmic distribution or discretization.

Cross Validation : The cross validation let the solver to generalize better since the cross validation divide the train dataset to the number of folds so every part of the dataset is used for the training.

Preprocessing : By way of preprocessing, I use a `ColumnTransformer` with in argument `OneHotEncoder` to transform the categorical variables. To find out which features are categorical, the `cat_indicator` variable take part.

[5] `https://benchopt.github.io/`

Optuna : To select hyperparameters, I use in this benchmark the optimization package Optuna. This allow me to specify a value range which can be logarithmic or discretized. The advantage of Optuna is to make *trials* that adjust hyperparameters at every new trials in order to find the best combination.

SufficientProgressCriterion : Compared to the original benchmark, `SingleRunCriterion()` has been replaced by `SufficientProgressCriterion()` to adopt a `callback` strategy. The `callback` allows multiple iterations for the solver to increase performances. A functionality of the solver is to verify the overfitting. The `callback` is set in a loop condition and returns `True` while it's needed to make a new iteration that can improve the performances.

AverageClassifier : This benchmark also required creating a *meta-estimator* similar to a `VotingClassifier`[6]. The reason to create the `AverageClassifier` is to determine from fitted solvers predictions of the cross validation, majority class for every sample. This avoid to fit again at each new iteration.

Metrics

Metrics show another point of view than the train and test score. Within the scope of the classification, the goal is to check the predictions done by the solvers.

Balanced accuracy : The `balanced_accuracy` returns a score obtained by calculate the average of recall values of each class.

ROC AUC : This metric compute the area under the ROC curve. By activating `ovr` – One Versus Rest – parameter, it compares the area under the ROC curve with those of other classes.

[6] `https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.VotingClassifier.html`

Solvers

Still with the aim to compare my results with those of Grinsztajn (2022), I added the same decision trees solvers than those of the paper.

HistGradientBoostingClassifier : As hyperparameters, this solver takes `max_iter` which decides the number of trees to use. Then `learning_rate` adjust how the next tree have to learn about errors of the previous one. Finally, `l2_regularization` perform a L2 regularization.

XGBoost : `XGBClassifier` has also the hyperparameters `learning_rate`, `l2_regularization` as well as `n_estimators`, set at same value range than those of `HistGradientBoostingClassifier`. `XGBoost` has in addition `max_depth` restricting the depth of every tree (i.e complexity).

RandomForest : `RandomForestClassifier` receive only `n_estimators` hyperparameter, determining the number of random trees to use.

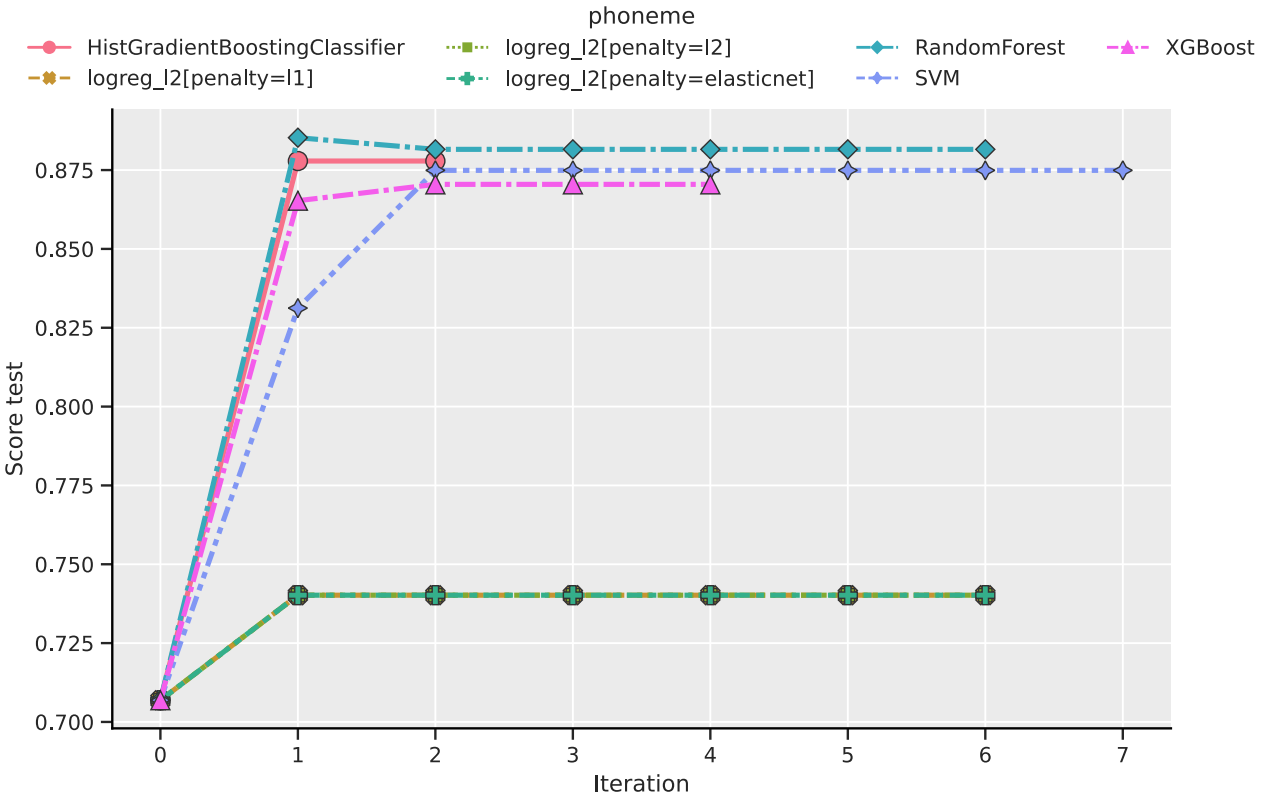
LogisticRegression : `LogisticRegression` is evaluated on tree penalties : lasso (L1), ridge (L2) and elasticnet. For each of these, `LogisticRegression` takes `C` as hyperparameter. This hyperparameter control the regularization. For the elasticnet penalty, I added `l1_ratio` hyperparameter.

SVC : For `SVC`, has as hyperparameters `C` and `kernel`. The first one is a parameter of regularization that limits the importance of each point. The second one allows to choose a type of kernel among `rbf`, `poly` or `linear`. With `rbf` and `poly`, a third hyperparameter called `gamma` take part and allows to control the kernel radius.

Benchmark results

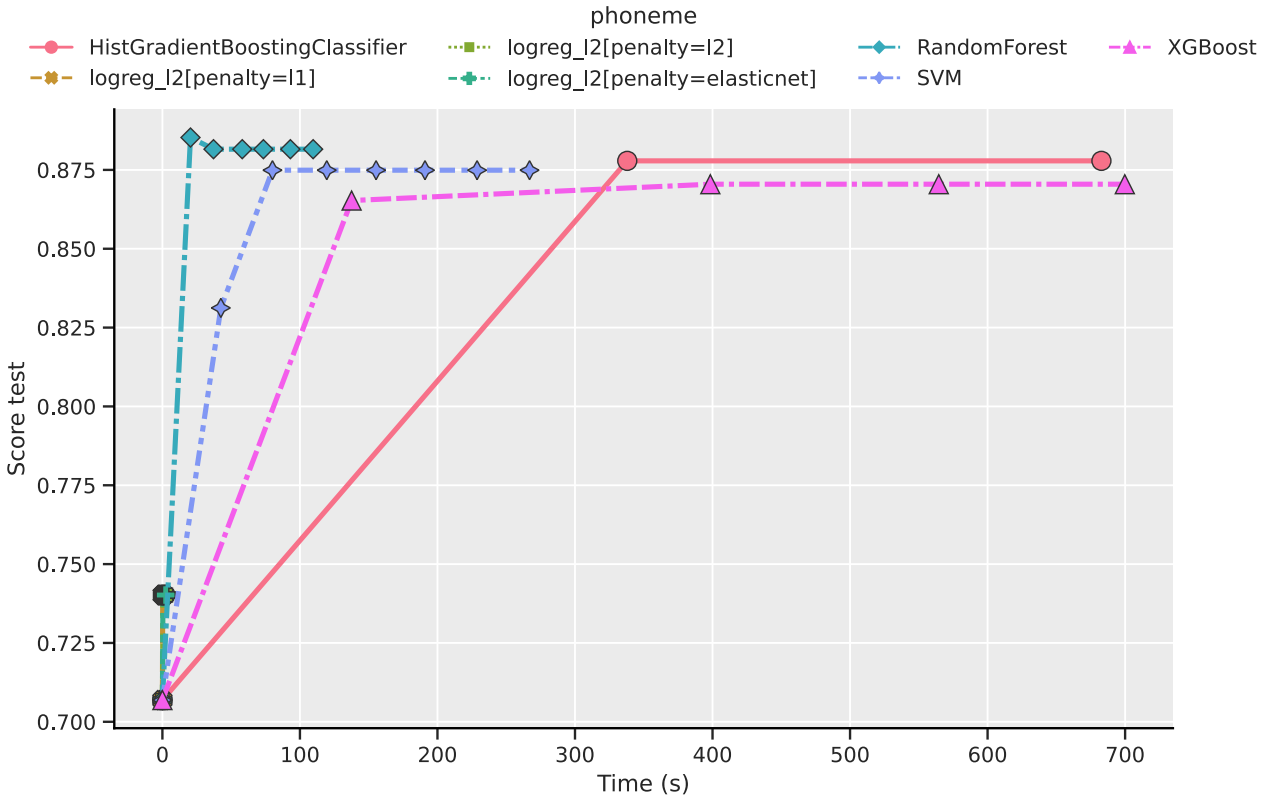
The following results have bee achieved with the three datasets of Grinsztajn (2022) paper. It's the datasets `phoneme`, `wine` and `kdd_ipums_la_97-small` that contains less than 10000 samples. The `wine` dataset modeling the quality of a portuguese wines from physicochemical characteristics. Then the goal of the `phoneme` dataset is to distinguish between nasal and oral sounds. And well, the `kdd_ipums_la_97-small` dataset is about the census of the Los Angeles population converted to a binary case.
For each results, there are always a first point where all the solvers have the same value. That's the `score_test` value of `DummyClassifier`, which help to initialize the loop of the `callback`.

phoneme



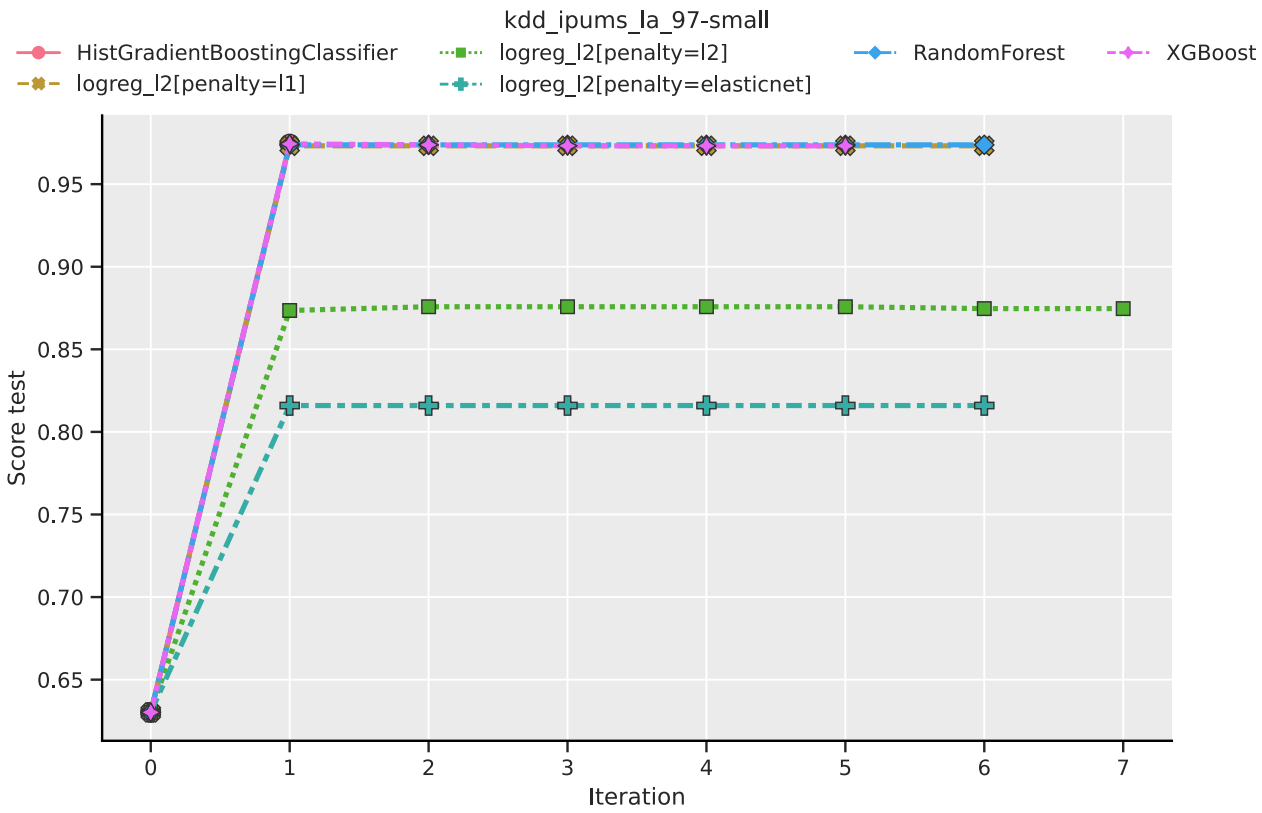
This first figure shows the number of iterations executed by solvers to suit the score obtained on test data. For this dataset, the solvers based on gradient methods for decision trees have the fewest iterations (i.e they reached the timeout). Conversely, `RandomForest` seems to be the best solver. It looks that random methods are better than gradient methods. Between `HistGradientBoostingClassifier` and `XGBoost`, there is `SVM`, obtaining a very good score as well as the most iterations. The

last is `LinearRegresssion` having the worst score on every penalty. A hypothesis can be that the data is not very linear.



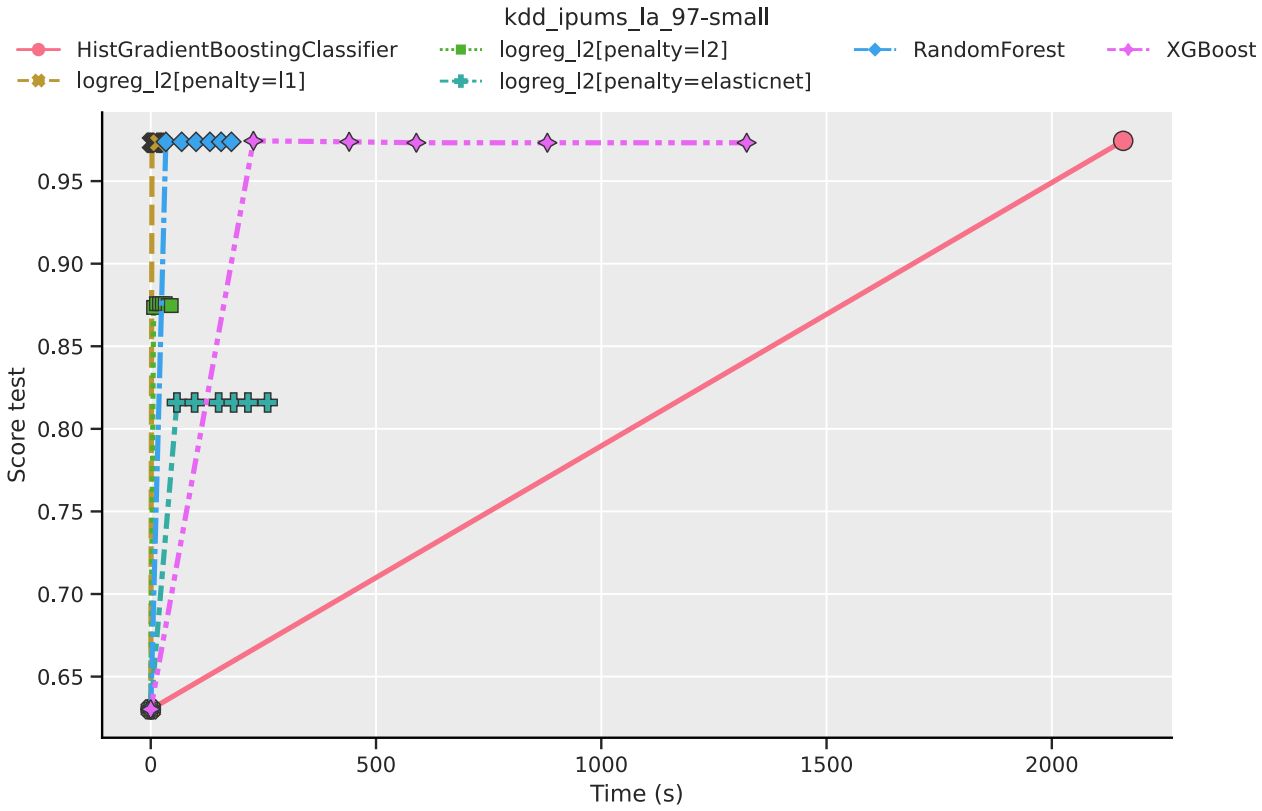
Now looking at the time took by the solver. `RandomForest` has the best performances and is the best solver for this dataset. The `LogisticRegression` is quick but doesn't have a good score. `SVM` also perform well but takes more time than `RandomForest` – due to the number of samples. As stated in the last paragraph, gradient methods takes so much time for this dataset. However, gradient methods obtain similar test score to `SVM`.

kdd_ipums_la_97-small

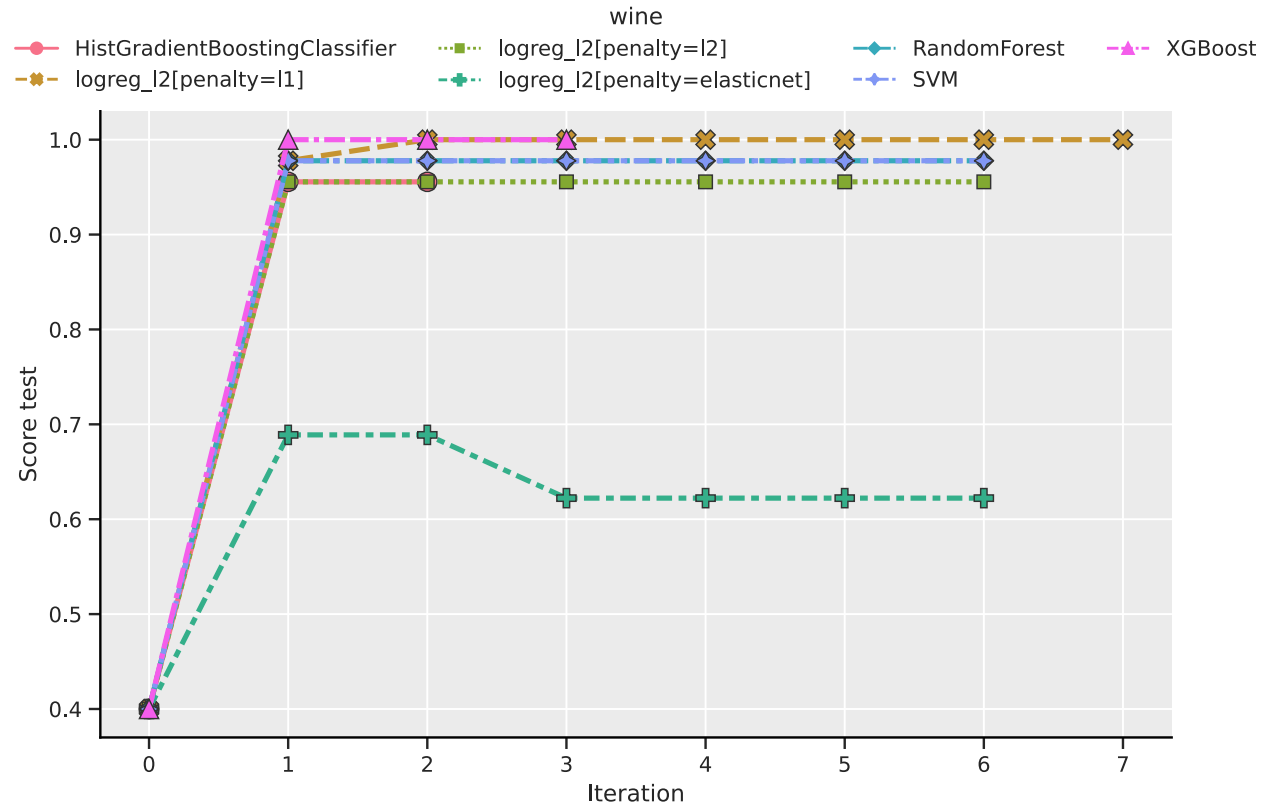


For this datasets, all solvers have an excellent score excpet for the `LinearRegression` with penalty L2 and elasticnet. Only L1 penalty fall into the line of other solvers score. Some solvers makes more iterations than others but the score variations are not meaningful.
Here, the solver `SVM` is not executed because of the large number of samples – 5188 samples –, it takes too much time to have satisfactory results.

Contrary to the previous dataset, `RandomForest` is not the best solver. `LinearRegression` with L1 penalty L1 is the most efficient solver for this dataset. Again, decisions trees using gradient methods are slower than `RandomForest`. By the way, `HistGradientBoostingClassifier` only has two iterations, although `timeout` time has been increased for this solver. It seems to take a lot of time to compute but obtain at least a very good score.

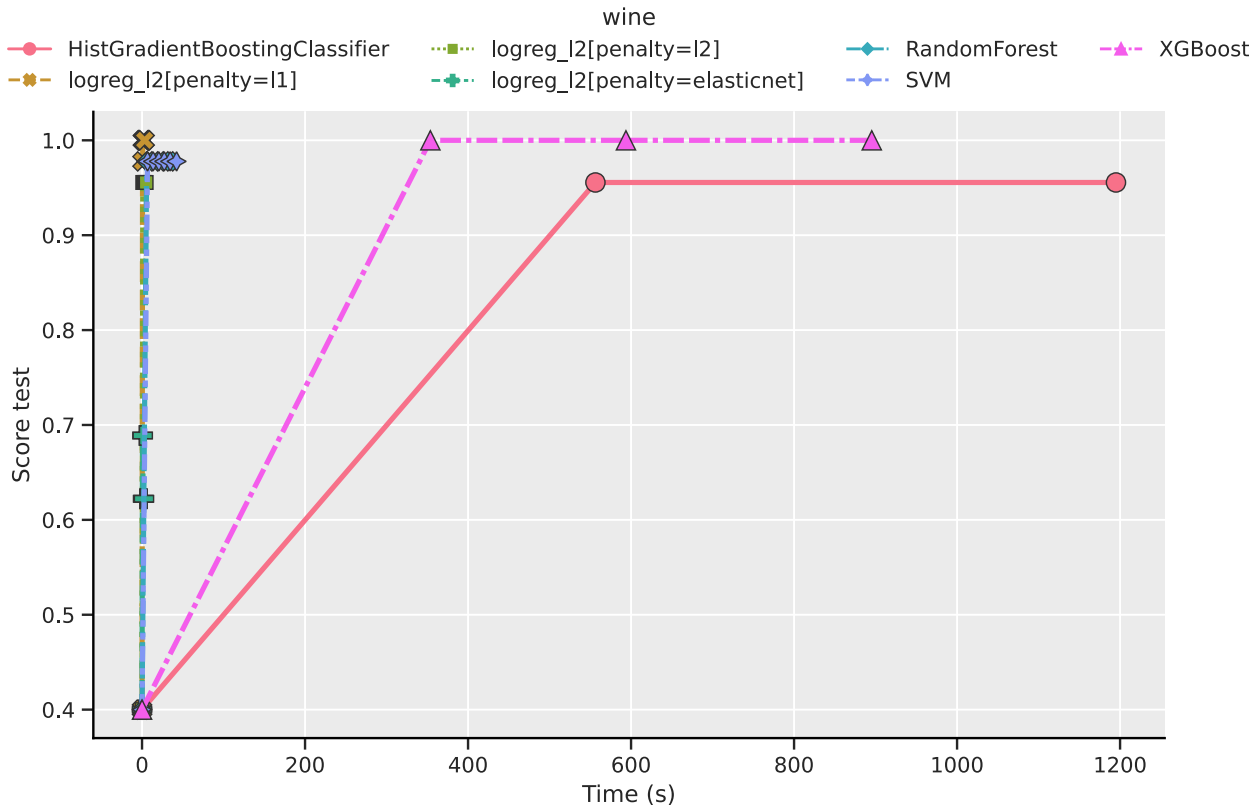


wine



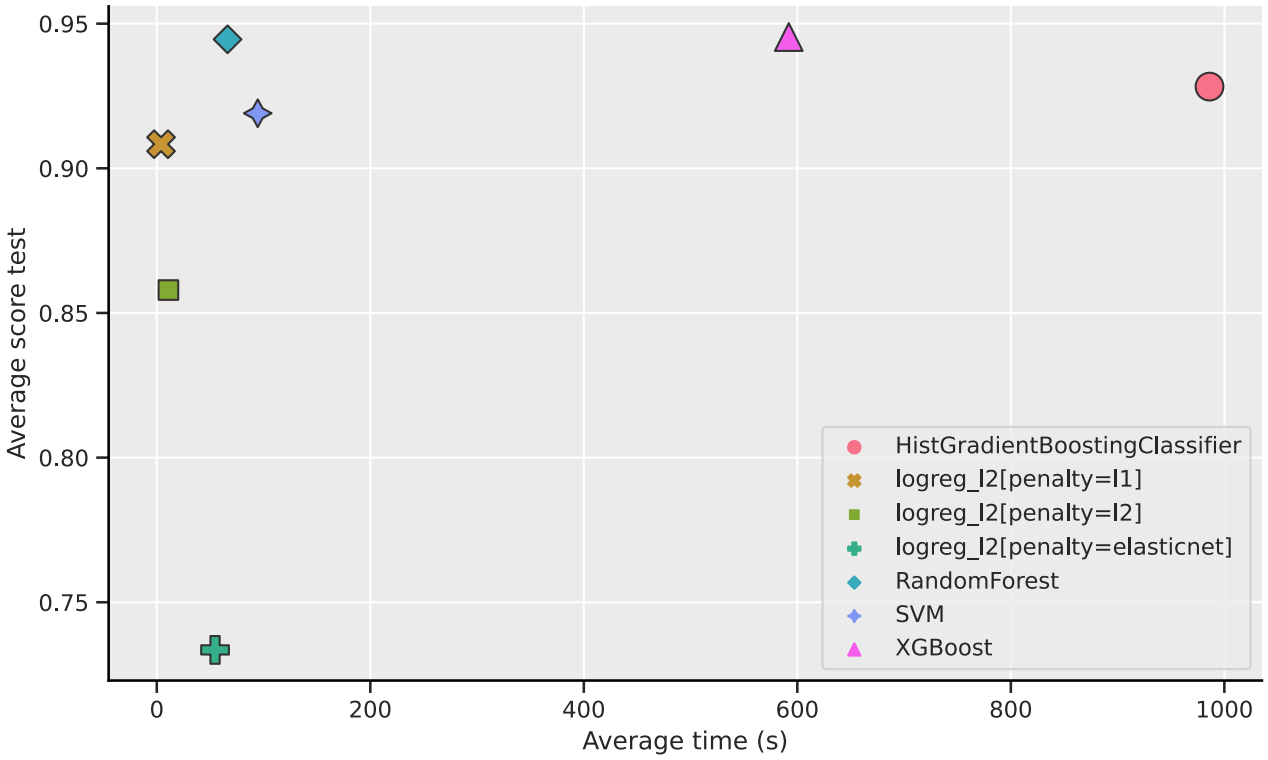
`LinearRegression` has never been efficient on previous datasets but in this last one, `LinearRegression` has not a good score with elasticnet only. The test score

reduce after three iterations, that shows a situation of overfitting. With L1 penalty, `LinearRegression` converge slower than `XGBoost` but reach quickly the same score as the latter. The score value is 1 for these two solver, that can lead to a very good learning for this datasets – confirmed by the value of 1 of the train score. For the L2 penalty, the test score is the same as `HistGradientBoostingClassifier`. This time, `RandomForest` is not above the decisions tree based on gradient methods. `RandomForest` has also the same score and number of iterations as `SVM`.



In a general way, all solvers are really fast for this dataset, except decisions tree based on gradient methods. `LinearRegression` with L1 penalty seems to be the best solver for this dataset.

Average over datasets



This last figure shows the average time taken in relation to average test scores. `LinearRegression` is the fastest solver and it's with L1 penalty that the results are the best. As noticed with the tree dataset, decisions trees based on gradient are the slowest but get very good scores. Finally, `SVM` and `RandomForest` are the solver the most efficient and in relation to the average time taken and the average test score, `RandomForest` is the most efficient solver on these datasets.

Bibliography

Léo Grinsztajn. Why do tree-based models still outperform deep learning on typical tabular data?, 11 2022. URL `https://inria.hal.science/hal-03723551v2`.